Universidad Mariano Gálvez
de Guatemala

Compatibilidad entre consumidor y proveedor utilizando Pact

Contract Testing con Pact · Informe académico

Responsable: Jefry Ariel Mas Monterroso

Caso de estudio: consulta de productos mediante una API HTTP en TypeScript.

Fecha de ejecución: 29/10/2026 · America/Guatemala.

Versión de entrega: entrega-v1.0.0

Repositorio del proyecto

Código, contrato y evidencias de esta entrega

Documento generado localmente y preparado para cargar en Canvas, la plataforma educativa. No se ha realizado la carga al aula virtual.

1. Introducción y contexto del caso

Cuando dos aplicaciones se comunican, una respuesta HTTP exitosa no garantiza que ambas interpreten los datos de la misma manera. Un campo puede conservar su nombre y cambiar de tipo, provocando un error en el consumidor. Las pruebas de contrato permiten expresar las expectativas del consumidor y comprobarlas contra el proveedor.

El objetivo de este proyecto es comprobar de forma reproducible la compatibilidad de un consumidor de productos con su API proveedora. Se implementaron ambas partes en TypeScript y se utilizó Pact JS para generar y verificar un contrato local, sin depender de un Broker ni de servicios pagados.

Interacción 1: producto existente

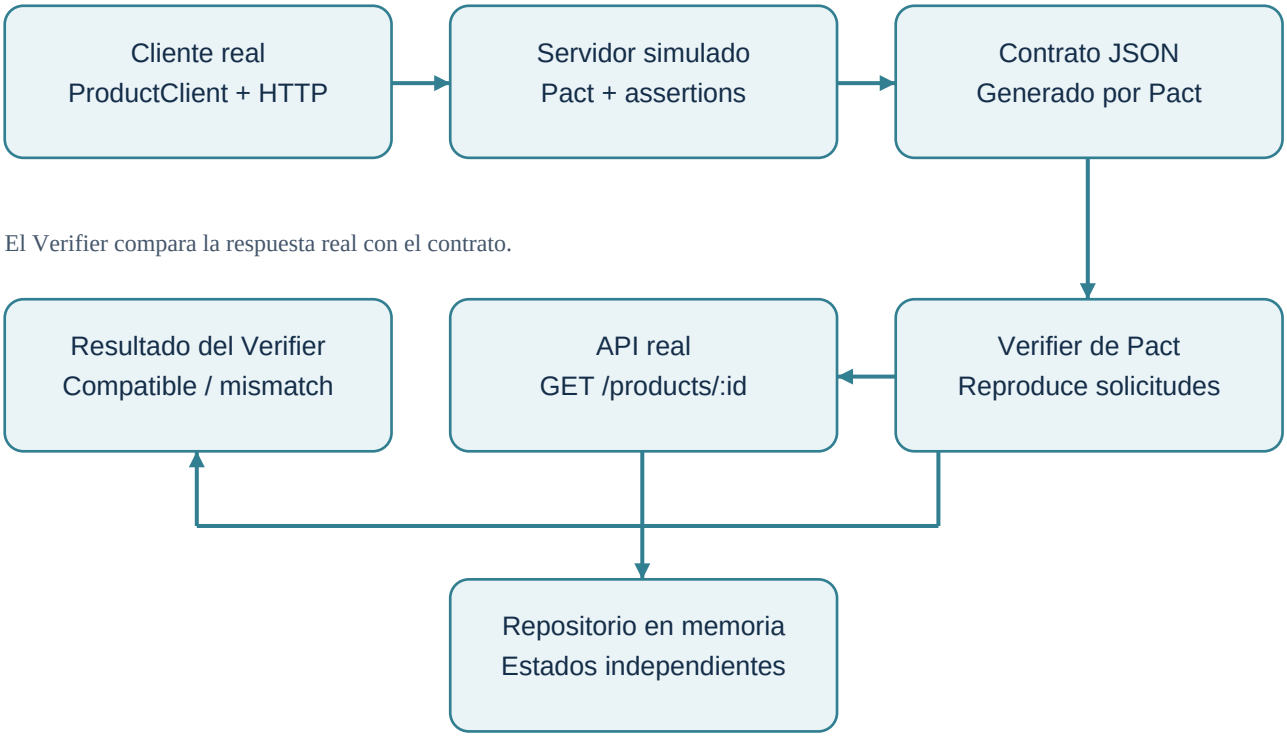
Precondición: existe p-100 en el catálogo. Solicitud: GET /products/p-100 con Accept: application/json. Respuesta: HTTP 200 y un producto con id p-100, name de texto, price decimal y available booleano. El consumidor devuelve kind: found y valida los tipos antes de utilizar los datos. Esta interacción representa el flujo principal de consulta.

Interacción 2: producto inexistente

Precondición: el catálogo no contiene p-404. Solicitud: GET /products/p-404. Respuesta: HTTP 404 con error.code igual a PRODUCT_NOT_FOUND y error.message de texto. El consumidor devuelve kind: not-found. Se eligió este caso porque una ausencia esperada debe distinguirse de un fallo del servidor o de una ruta equivocada.

Las dos respuestas utilizan Content-Type: application/json; charset=utf-8. Las rutas, estados HTTP, identificador y código de error son valores exactos; los campos variables se comprueban con matchers de tipo. La muestra exitosa es un teclado con precio Q249.90 y disponibilidad verdadera.

2. Arquitectura y herramientas



ProductClient interpreta los resultados y utiliza HttpClient, un cliente HTTP reutilizable basado en fetch. La API usa node:http y consulta ProductRepository. Durante las pruebas del consumidor, Pact sustituye a la API; durante la verificación, el verificador envía solicitudes a la API real y compara las respuestas con el archivo JSON generado previamente.

Herramienta	Versión / propósito
Node.js / npm	22.23.2 / 10.9.8; ejecución y dependencias
TypeScript / tsx	5.9.3 / 4.23.15; tipado estricto y ejecución
Pact JS	17.1.4; contrato de especificación Pact 3.0.0
node:test / assert	Pruebas y assertions incluidas con Node.js
Playwright / Python	1.64.1 / 3.13.14; capturas e informe
ReportLab / pypdf	4.5.1 / 6.21.2; generación y revisión del PDF

3. Diseño y ejecución de las pruebas

Consumidor y generación automática

tests/consumer.pact.test.ts configura PactV3 y sus dos interacciones. executeTest inicia el servidor simulado y ejecuta el ProductClient y HttpClient reales contra su URL. Las assertions comparan los resultados completos. Pact verifica las solicitudes y escribe pacts/ProductConsumer-ProductProvider.json; este JSON no se redacta manualmente.

Matchers relevantes

string conserva el tipo de name y del mensaje; decimal exige un número decimal para price; boolean valida available. El id y PRODUCT_NOT_FOUND se fijan por igualdad para evitar que otro producto u otra clase de error sean aceptados como correctos.

Proveedor y estados independientes

tests/verify-provider.ts inicia la misma API usada por la aplicación. Los stateHandlers reemplazan por completo el catálogo: [sampleProduct] para existencia y [] para ausencia. Cada ejecución dispone de un repositorio propio y un puerto libre. Una prueba adicional comprueba las consultas en ambos órdenes y el servidor se cierra incluso si la verificación falla.

Componentes reales y controlados

Son reales el consumidor, fetch, la comunicación HTTP, la API, las rutas y la serialización. En las pruebas del consumidor la API se simula con Pact. En el proveedor se controlan los datos del repositorio en memoria, que es la persistencia real de esta aplicación; no se simula una base de datos externa.

Las cinco pruebas complementarias comprueban HTTP 500, un 404 de otra causa, un precio de tipo incorrecto, un producto con otro ID y la independencia de estados. Estas pruebas complementan las dos interacciones contractuales.

```
npm ci
npm test
npm run demo
npm run evidence:capture -- --edge
npm run report
```

Para capturas sin Edge: npx playwright install chromium; luego npm run evidence:capture. Para reconstruir el PDF: python -m pip install -r requirements-report.txt. El README describe instalación y ejecución completa.

4. Evidencia: consumidor

Se ejecutaron las dos pruebas con el cliente HTTP real contra el servidor simulado de Pact. Se esperaba interpretar correctamente el producto y el error estructurado. El resultado fue 2 pruebas aprobadas, 0 fallidas y salida 0; Pact generó automáticamente el contrato.



Consumidor: dos pruebas aprobadas

Captura del visor de registros originales de una ejecución real. No es una terminal nativa.

29/10/2026, 19:38:32 · America/Guatemala · v22.23.2

Código de salida real: 0 · esperado: 0

```
node --import tsx --test --test-concurrency=1 tests/consumer.pact.test.ts
```

```
TAP version 13
# Subtest: producto existente: cliente real interpreta HTTP 200 y sus tipos
ok 1 - producto existente: cliente real interpreta HTTP 200 y sus tipos
  ---
  duration_ms: 36.004
  type: 'test'
  ...
# Subtest: producto inexistente: cliente real distingue HTTP 404 estructurado
ok 2 - producto inexistente: cliente real distingue HTTP 404 estructurado
  ---
  duration_ms: 10.9736
  type: 'test'
  ...
1..2
# tests 2
# suites 0
# pass 2
# fail 0
# cancelled 0
# skipped 0
# todo 0
# duration_ms 1324.8281
```

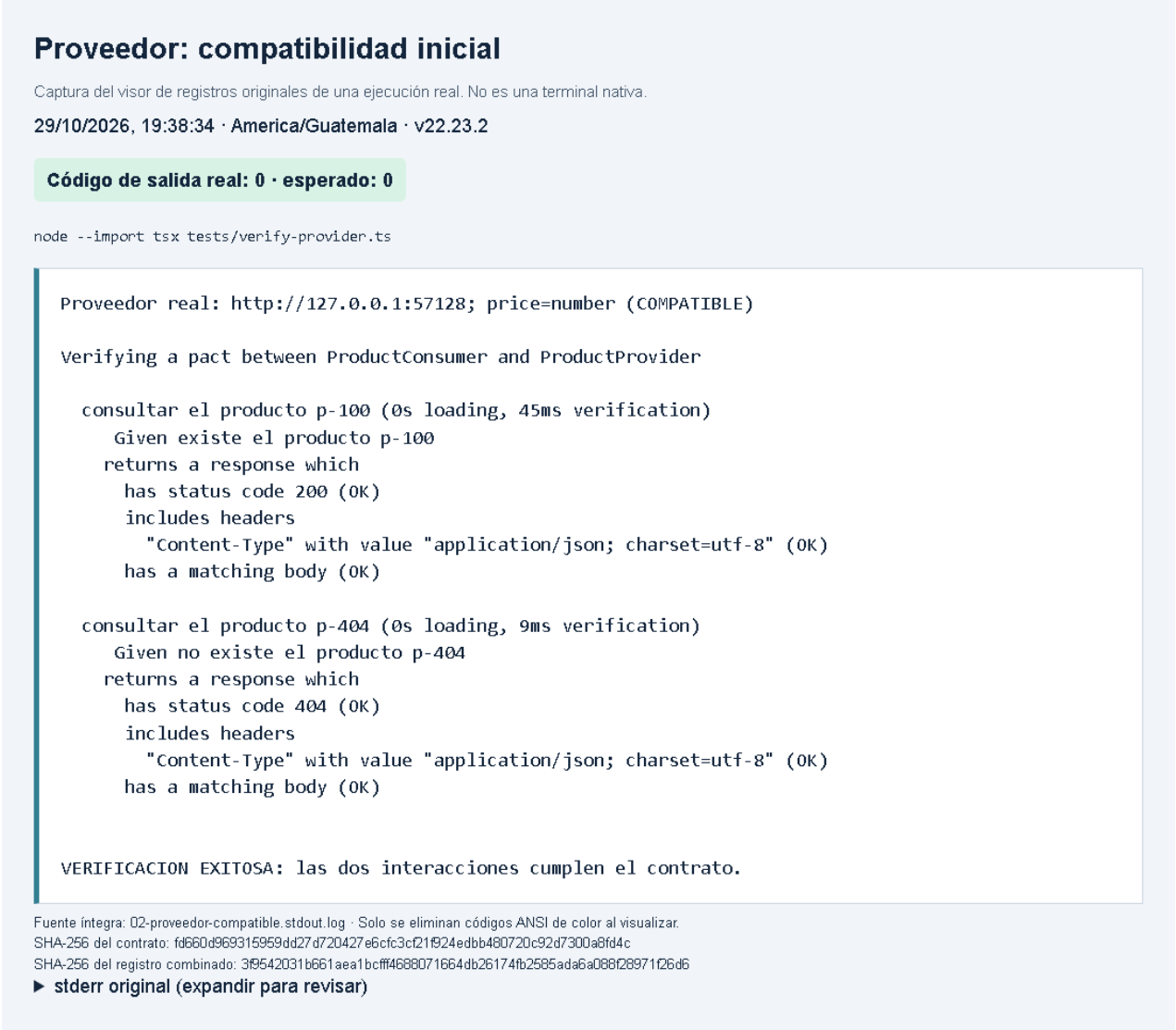
Fuente íntegra: 01-consumidor.stdout.log · Solo se eliminan códigos ANSI de color al visualizar.
SHA-256 del contrato: fd660d969315959dd27d720427e6cfc3cf21f924edbb480720c92d7300a8fd4c
SHA-256 del registro combinado: e30d5cab9d72135468c270bbff21eb4635a2e0c51d9b24687fd9cdeca76300aa

▶ stderr original (expandir para revisar)

Captura auténtica del visor de stdout original; no es una terminal nativa. stderr completo se conserva en los registros del repositorio.

5. Evidencia: proveedor compatible

Se verificó el contrato generado contra la API real con sus estados independientes. Se esperaba cumplir ambos acuerdos. Pact confirmó los estados 200 y 404, los encabezados y los cuerpos de respuesta. La ejecución terminó con salida 0.



Proveedor: compatibilidad inicial

Captura del visor de registros originales de una ejecución real. No es una terminal nativa.

29/10/2026, 19:38:34 · America/Guatemala · v22.23.2

Código de salida real: 0 · esperado: 0

```
node --import tsx tests/verify-provider.ts
```

```
Proveedor real: http://127.0.0.1:57128; price=number (COMPATIBLE)

Verifying a pact between ProductConsumer and ProductProvider

  consultar el producto p-100 (0s loading, 45ms verification)
     Given existe el producto p-100
    returns a response which
      has status code 200 (OK)
      includes headers
        "Content-Type" with value "application/json; charset=utf-8" (OK)
      has a matching body (OK)

  consultar el producto p-404 (0s loading, 9ms verification)
     Given no existe el producto p-404
    returns a response which
      has status code 404 (OK)
      includes headers
        "Content-Type" with value "application/json; charset=utf-8" (OK)
      has a matching body (OK)


VERIFICACION EXITOSA: las dos interacciones cumplen el contrato.
```

Fuente íntegra: 02-proveedor-compatible.stdout.log · Solo se eliminan códigos ANSI de color al visualizar.
SHA-256 del contrato: fd660d969315959dd27d720427e6cfc3cf21f924edbb480720c92d7300a8fd4c
SHA-256 del registro combinado: 3f9542031b661aea1bcfff4688071664db26174fb2585ada6a088f28971f26d6

▶ stderr original (expandir para revisar)

Captura auténtica del visor de stdout original; no es una terminal nativa. stderr completo se conserva en los registros del repositorio.

6. Evidencia: incompatibilidad real

Se activó --incompatible: el proveedor serializó price como texto. Se esperaba que Pact rechazara ese tipo. La interacción p-100 falló en $.price: esperaba decimal y recibió String. La interacción 404 siguió aprobada. El proceso devolvió 1 y el contrato no cambió.



Proveedor: incompatibilidad detectada

Captura del visor de registros originales de una ejecución real. No es una terminal nativa.

29/10/2026, 19:38:35 · America/Guatemala · v22.23.2

Código de salida real: 1 · esperado: 1

```
node --import tsx tests/verify-provider.ts --incompatible
```

```
Proveedor real: http://127.0.0.1:57136; price=string (INCOMPATIBLE)

Verifying a pact between ProductConsumer and ProductProvider

  consultar el producto p-100 (0s loading, 45ms verification)
     Given existe el producto p-100
    returns a response which
      has status code 200 (OK)
      includes headers
        "Content-Type" with value "application/json; charset=utf-8" (OK)
      has a matching body (FAILED)

  consultar el producto p-404 (0s loading, 9ms verification)
     Given no existe el producto p-404
    returns a response which
      has status code 404 (OK)
      includes headers
        "Content-Type" with value "application/json; charset=utf-8" (OK)
      has a matching body (OK)


Failures:

1) Verifying a pact between ProductConsumer and ProductProvider Given existe el producto p-100 -
consultar el producto p-100
    1.1) has a matching body
           $.price -> Expected '249.9' (String) to be a decimal number

There were 1 pact failures

[19:38:36.659] ERROR (19928): pact-core@20.2.0: Verification unsuccessful
```

Fuente íntegra: 03-proveedor-incompatible.stdout.log · Solo se eliminan códigos ANSI de color al visualizar.
SHA-256 del contrato: fd660d969315959dd27d720427e6cfc3cf21f924edbb480720c92d7300a8fd4c
SHA-256 del registro combinado: 0ebc6cb4edbd0eaf8cc4dc6d91c6a140b34f910790b43aa47b92ee9c630c69bc

▶ stderr original (expandir para revisar)

Captura auténtica del visor de stdout original; no es una terminal nativa. stderr completo se conserva en los registros del repositorio.

7. Evidencia: compatibilidad restaurada

Se inició otra instancia sin la mutación, con price numérico. Se esperaba recuperar la compatibilidad. Pact aprobó nuevamente las dos interacciones y devolvió 0. La versión normal y la entrega final conservan el proveedor compatible.



Proveedor: compatibilidad restaurada

Captura del visor de registros originales de una ejecución real. No es una terminal nativa.

29/10/2026, 19:38:36 · America/Guatemala · v22.23.2

Código de salida real: 0 · esperado: 0

```
node --import tsx tests/verify-provider.ts
```

```
Proveedor real: http://127.0.0.1:57144; price=number (COMPATIBLE)

Verifying a pact between ProductConsumer and ProductProvider

  consultar el producto p-100 (0s loading, 42ms verification)
     Given existe el producto p-100
    returns a response which
      has status code 200 (OK)
      includes headers
        "Content-Type" with value "application/json; charset=utf-8" (OK)
      has a matching body (OK)

  consultar el producto p-404 (0s loading, 8ms verification)
     Given no existe el producto p-404
    returns a response which
      has status code 404 (OK)
      includes headers
        "Content-Type" with value "application/json; charset=utf-8" (OK)
      has a matching body (OK)


VERIFICACION EXITOSA: las dos interacciones cumplen el contrato.
```

Fuente íntegra: 04-proveedor-restaurado.stdout.log · Solo se eliminan códigos ANSI de color al visualizar.
SHA-256 del contrato: fd660d969315959dd27d720427e6cfc3cf21f924edbb480720c92d7300a8fd4c
SHA-256 del registro combinado: dc657e3ae20885351938a3540b467797a918fbce2ad9527f80db0cc67d57a357

▶ stderr original (expandir para revisar)

Captura auténtica del visor de stdout original; no es una terminal nativa. stderr completo se conserva en los registros del repositorio.

8. Reproducción, conclusiones y referencias

Demostración y restauración

npm run demo ejecuta las cuatro etapas y acepta únicamente las salidas 0, 0, 1 y 0. La etapa incompatible exige un BodyMismatch en $.price; no basta un fallo cualquiera. El cambio solo existe en la instancia de demostración. npm run test:provider restaura la compatibilidad al crear un proveedor sin esa opción, y npm test ejecuta la validación final completa.

Trazabilidad de la evidencia

resumen.json conserva fechas UTC, comandos, salidas y hashes SHA-256. Las capturas muestran la hora local de Guatemala. Los canales stdout y stderr se preservan por separado; el log combinado concatena ambos y no afirma su orden de intercalación. El visor elimina solo códigos de color ANSI para facilitar la lectura. El mismo contrato se utilizó en las cuatro etapas.

```
SHA-256 del contrato: fd660d969315959dd27d720427e6cfc3
cf21f924edbb480720c92d7300a8fd4c
```

Conclusiones

La verificación mostró que el consumidor y el proveedor son compatibles en los dos escenarios definidos. El fallo controlado demostró que conservar HTTP 200 no es suficiente: cambiar el tipo de un campo requerido rompe el acuerdo. Los estados independientes permitieron repetir las pruebas y recuperar el resultado correcto sin editar el contrato. Pact complementa las pruebas de comportamiento, pero este alcance no incluye carga, seguridad ni otros casos de negocio.

La entrega incluye código, contrato generado, cuatro capturas, registros originales y README reproducible. El informe está listo para cargar manualmente en Canvas.

Repositorio: jefroMMM/Contract-Testing-con-Pact
Versión final: entrega-v1.0.0

Pact JS: pruebas del consumidor

Pact JS: verificación del proveedor

Pact: estados del proveedor

Pact JS 17.1.4: requisitos del paquete